



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Registro de asistencia estudiantil

Integrantes:

Rodrigo Escudero

Mateo Cueto

Gonzalo Leal

Diego Binimelis

9 de mayo de 2026

Índice

Lista de figuras	II
Lista de tablas	III
1 ¿Qué datos maneja el sistema? (SIA-1)	2
2 Las clases que construyen el sistema (SIA-2)	3
3 Buenas prácticas (SIA-3)	9
4 Uso de colecciones de Java (SIA-4)	10
5 Sobrecarga de métodos (SIA-5)	12
6 Sobreescritura y polimorfismo (SIA-6)	13
7 Insertar y consultar datos del sistema (SIA-7)	14
8 Modificar, eliminar y buscar información (SIA-8)	16
9 Funcionalidad propia del programa (SIA-9)	17
10 Consola o ventanas, a eleccion (SIA-10)	20
11 Los datos no se pierden al cerrar (SIA-11)	23
12 Manejando errores sin que el programa se caiga (SIA-12)	24
13 Control de versiones con GitHub (SIA-13)	26
14 Aspectos opcionales implementados	27

Índice de figuras

Figura 2.1	Diagrama de Clases del sistema 1: Clases de Control y Exception ...	4
Figura 2.2	Diagrama de Clases del sistema 2: Clases principales del Sistema ...	5
Figura 2.3	Diagrama de Clases del sistema 3: Clase Asistencia y clases hijas ...	5
Figura 2.4	Diagrama de Clases del sistema 4: Clase Ventana y referencia a JFrame	6
Figura 2.5	Diagrama de Clases del sistema 5: Clases de ventanas y referencia a JPanel	7
Figura 2.6	Diagrama de Clases del sistema 6: Clases de ventanas y referencia a JPanel	7
Figura 2.7	Diagrama de Clases Completo del sistema, aquí para ver con detalle	8
Figura 4.1	Diagrama de Estructuras Principales del sistema	10
Figura 4.2	Diseño del TreeMap listaGlobal de estudiantes	10
Figura 4.3	Diseño del ArrayList listaAsistencia	11
Figura 7.1	Vista previa del menú en modo ventana	15
Figura 9.1	Diagrama de flujo del proceso de ordenamiento y visualización de la lista de curso	18
Figura 10.1	Selector de modo de arranque	20
Figura 10.2	Vista del menu en modo consola	22
Figura 13.1	Commits realizados en Github	26

Índice de tablas

Tabla 9.1	Funcionalidades propias del negocio	17
Tabla 10.1	Diferencias y similitudes entre los modos de ejecución	21
Tabla 12.1	Excepciones personalizadas del sistema	24

Introducción

Este documento describe el desarrollo del **Sistema de Información de Asistencia Estudiantil**, una aplicación en Java que permite gestionar el registro de asistencia de alumnos en un colegio. El sistema maneja estudiantes, cursos y distintos tipos de asistencias (normal, inasistencia justificada, salida anticipada). Ofrece dos modos de interacción: consola (CLI) y ventanas (Swing), y guarda los datos en un archivo CSV para su persistencia. A continuación se detalla cómo cada componente del código cumple con los requisitos académicos SIA.

1. ¿Qué datos maneja el sistema? (SIA-1)

Para administrar la asistencia, necesitamos representar tres cosas principales:

- **Estudiante:** tiene un RUT (que lo identifica de forma única), nombre completo, edad y el curso al que pertenece.
- **Asistencia:** es un registro de un día específico. Guarda la fecha, una observación y, según el tipo, datos extra (por ejemplo, si fue puntual o el motivo de la falta).
- **Curso:** agrupa a los estudiantes del mismo nivel (por ejemplo, "1º Básico").

La relación entre ellos es sencilla: un **Curso** contiene varios **Estudiantes**, y cada **Estudiante** guarda una lista con todas sus **Asistencias** a lo largo del año.

2. Las clases que construyen el sistema (SIA-2)

A continuación se describen las clases Java que modelan los datos del dominio. Cada una tiene atributos y métodos que definen su comportamiento.

Clase Persona

Representa los datos básicos de cualquier persona en el sistema. Almacena nombre, apellidos, RUT y edad. Incluye un método `validarRut()` que comprueba el dígito verificador del RUT chileno mediante el algoritmo de módulo 11. Los setters `setRut()` y `setEdad()` lanzan automáticamente `RutInvalidoException` o `EdadInvalidaException` si los datos no son correctos, impidiendo que existan objetos con información inconsistente.

Clase Estudiante

Extiende a `Persona` añadiendo el curso al que pertenece y una lista de asistencias. Proporciona métodos para agregar, buscar, editar y eliminar registros de asistencia, así como para contar cuántos hay de cada tipo y calcular el porcentaje de asistencia mediante `getPorcentajeAsistencia()`. La lista de asistencias se retorna como una vista de solo lectura para proteger el encapsulamiento.

Jerarquía de Asistencia

`Asistencia` es una clase abstracta que define los atributos comunes a todo registro: ID, fecha y observación. A partir de ella se derivan tres subclases concretas:

- `AsistenciaNormal`: añade un indicador de puntualidad.
- `InasistenciaExtraordinaria`: añade el motivo de la falta.
- `SalidaAnticipada`: añade la hora en que el alumno se retiró.

¿Por qué es abstracta? En el dominio escolar nunca se registra una asistencia genérica: siempre corresponde a un tipo concreto (normal, inasistencia o salida anticipada). Declarar la clase como abstracta impide que se creen objetos sin tipo definido y, al declarar `getResumen()` como método abstracto, se obliga a cada subclase a proporcionar su propia descripción. Esto garantiza que el sistema siempre sepa exactamente qué ocurrió en cada jornada.

Clase Curso

Agrupar a los estudiantes de un mismo nivel académico. Utiliza internamente un `TreeMap` donde la clave es el RUT, manteniendo a los alumnos ordenados automáticamente. El método `getEstudiantes()` retorna un mapa inmodificable para evitar manipulaciones externas.

Clases de apoyo

- **Utilidades:** contiene métodos estáticos para validar el formato de fechas y comparar cronológicamente dos fechas.
- **GestorArchivos:** maneja la lectura y escritura de los archivos CSV que almacenan estudiantes y asistencias.

Excepciones personalizadas

El sistema define tres excepciones propias que extienden **Exception**:

- **EstudianteNoEncontradoException:** cuando un RUT no está en el mapa global.
- **RutInvalidoException:** cuando el RUT no pasa la validación del dígito verificador.
- **EdadInvalidaException:** cuando se intenta asignar una edad negativa o cero.

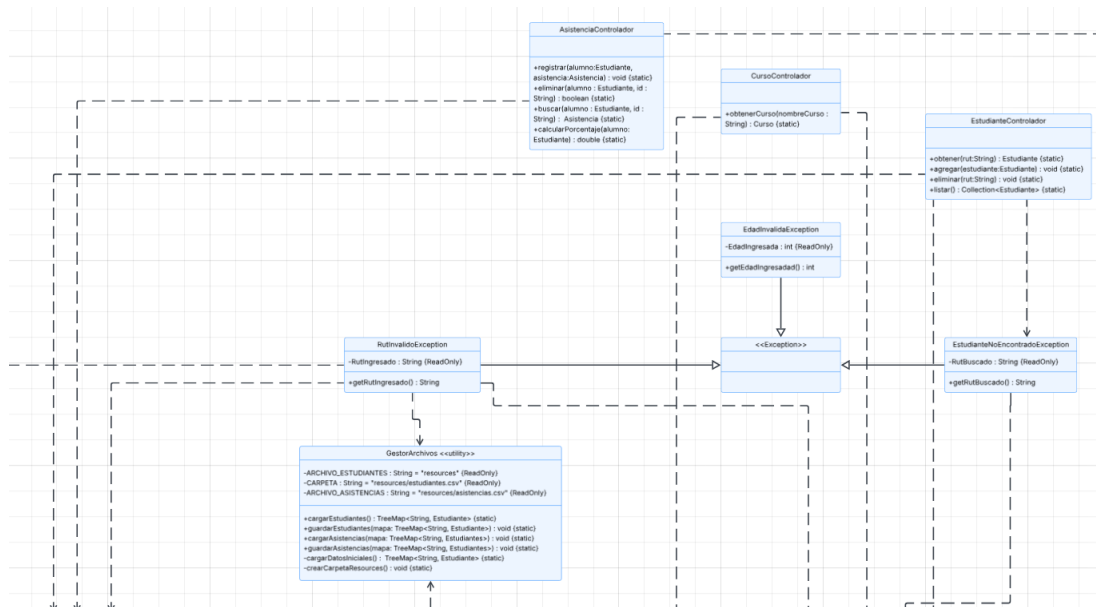


Figura 2.1: Diagrama de Clases del sistema 1: Clases de Control y Exception

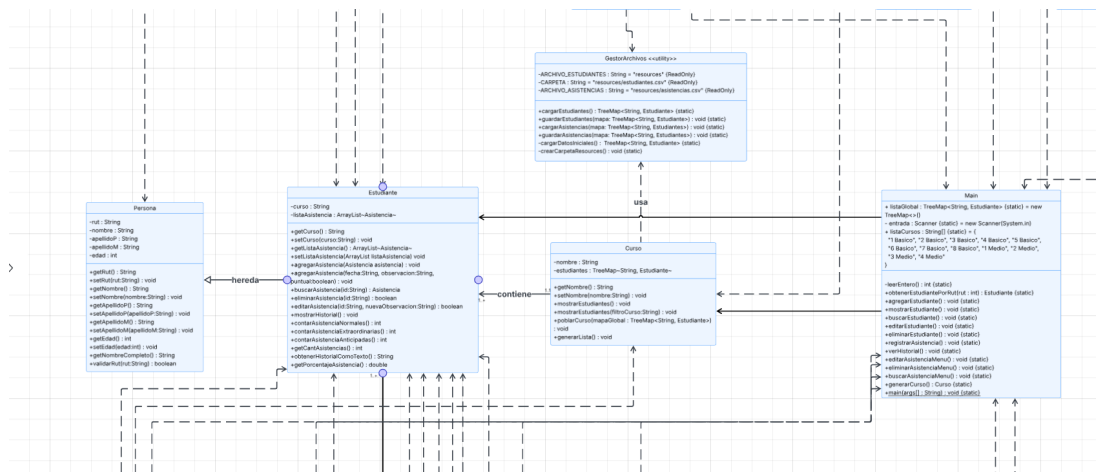


Figura 2.2: Diagrama de Clases del sistema 2: Clases principales del Sistema

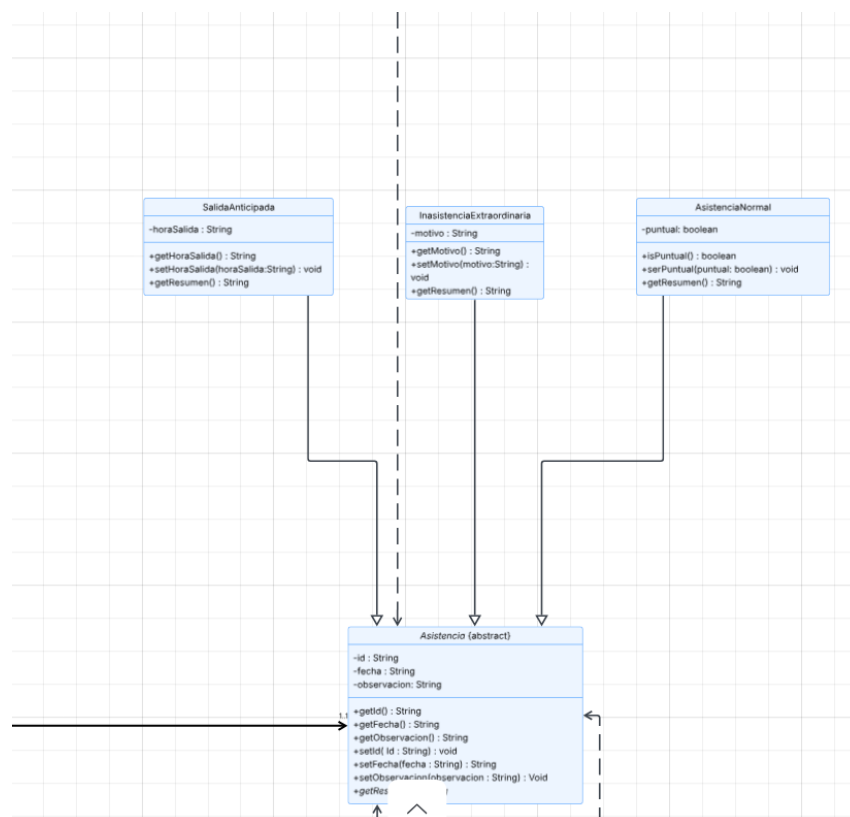


Figura 2.3: Diagrama de Clases del sistema 3: Clase Asistencia y clases hijas

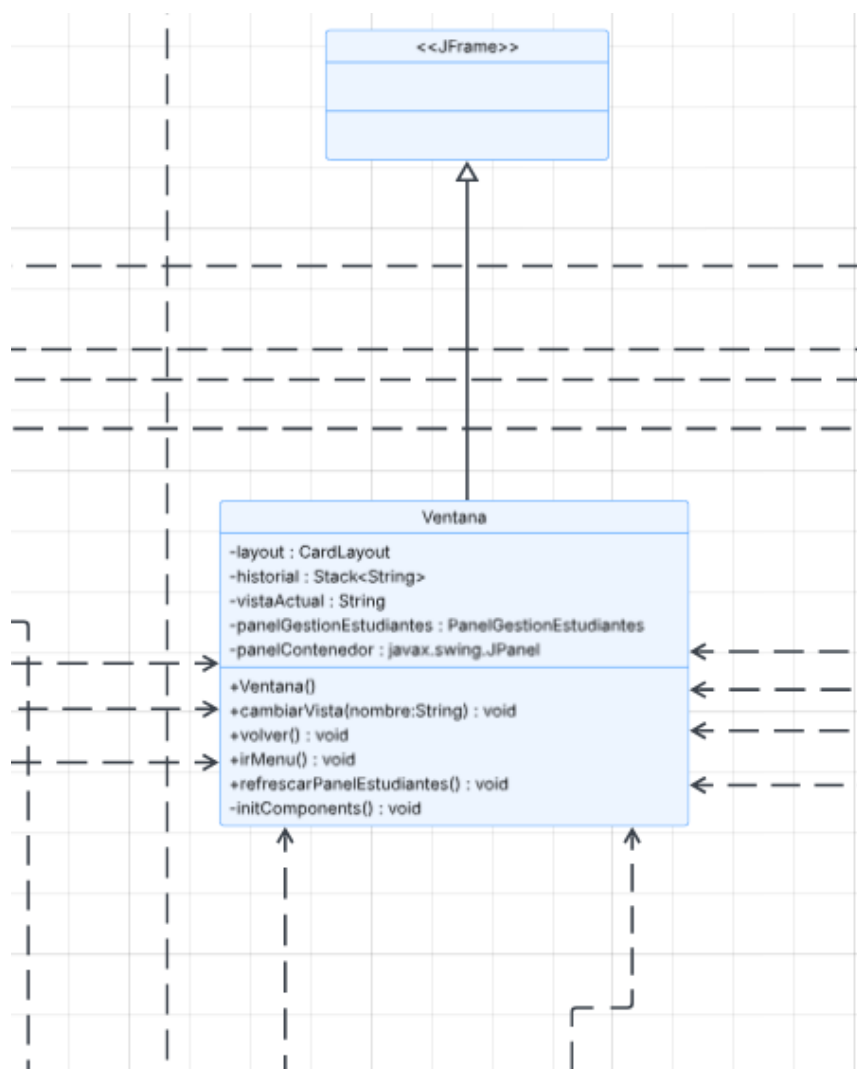


Figura 2.4: Diagrama de Clases del sistema 4: Clase Ventana y referencia a JFrame

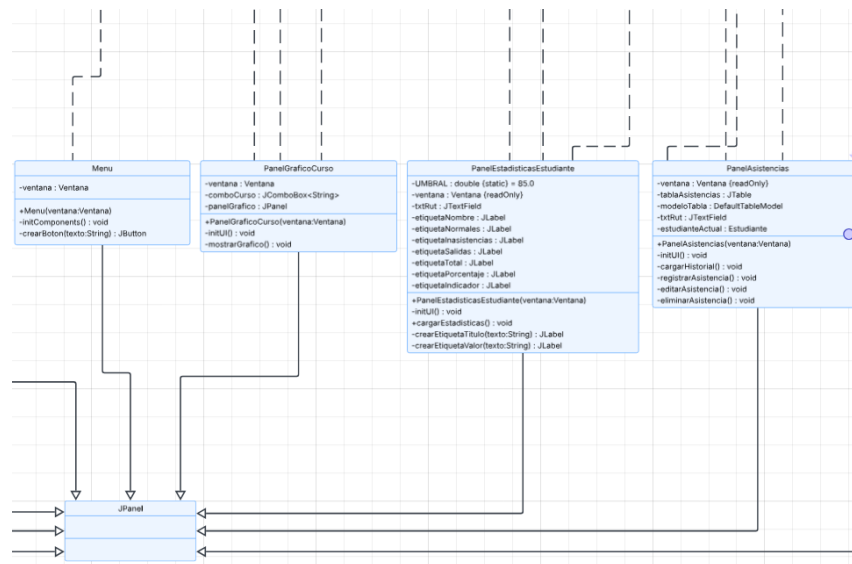


Figura 2.5: Diagrama de Clases del sistema 5: Clases de ventanas y referencia a JPanel

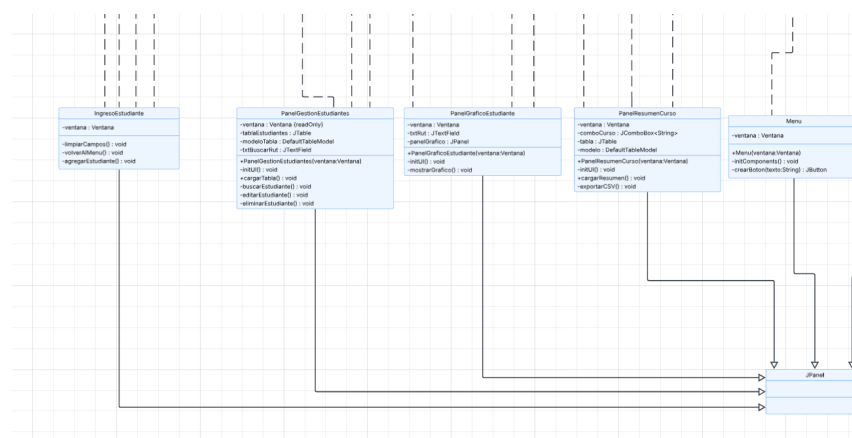
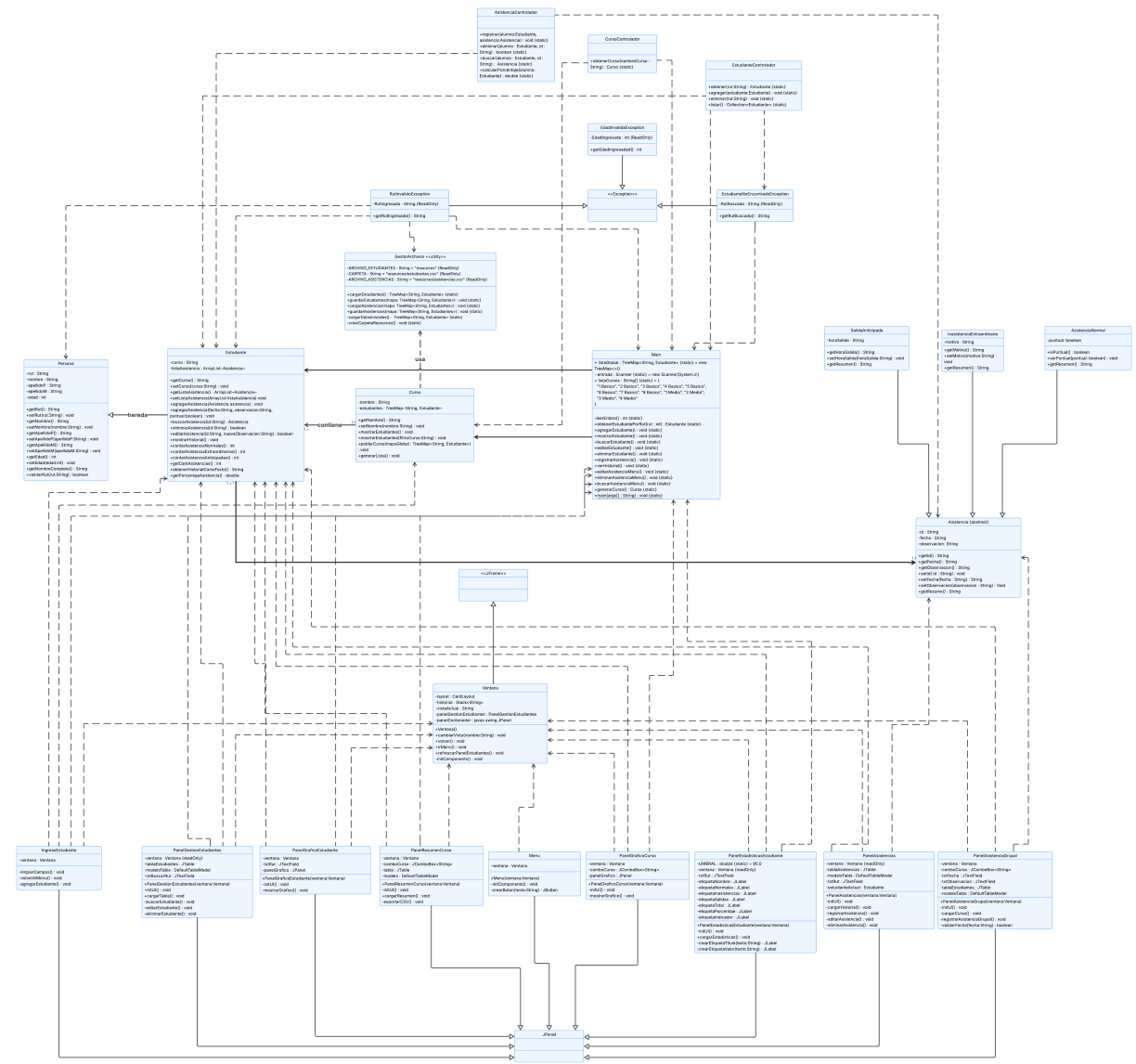


Figura 2.6: Diagrama de Clases del sistema 6: Clases de ventanas y referencia a JPanel



3. Buenas prácticas (SIA-3)

- **Atributos privados con getters y setters:** todas las clases del modelo declaran sus atributos como `private` y proporcionan métodos de acceso público. Además, métodos como `getListaAsistencia()` en `Estudiante` y `getEstudiantes()` en `Curso` retornan colecciones inmodificables.
- **Código modularizado en paquetes MVC:** el proyecto se organiza en tres paquetes que reflejan el patrón Modelo-Vista-Controlador:
 - **modelo:** clases de dominio (`Persona`, `Estudiante`, `Asistencia` y subclases, `Curso`, `GestorArchivos`) y excepciones.
 - **vista:** paneles Swing que conforman la interfaz gráfica.
 - **controlador:** clases intermediarias (`EstudianteControlador`, `AsistenciaControlador`, `CursoControlador`) que manejan la lógica del programa y evitan que las vistas accedan directamente al modelo.
- **Datos iniciales precargados:** la clase `GestorArchivos` incluye un método `cargarDatosIniciales()` que genera automáticamente estudiantes y asistencias de ejemplo si el archivo CSV no existe o está vacío, permitiendo demostrar las funcionalidades del sistema sin necesidad de ingresar datos manualmente.

4. Uso de colecciones de Java (SIA-4)

El sistema guarda la información en dos niveles de colecciones:

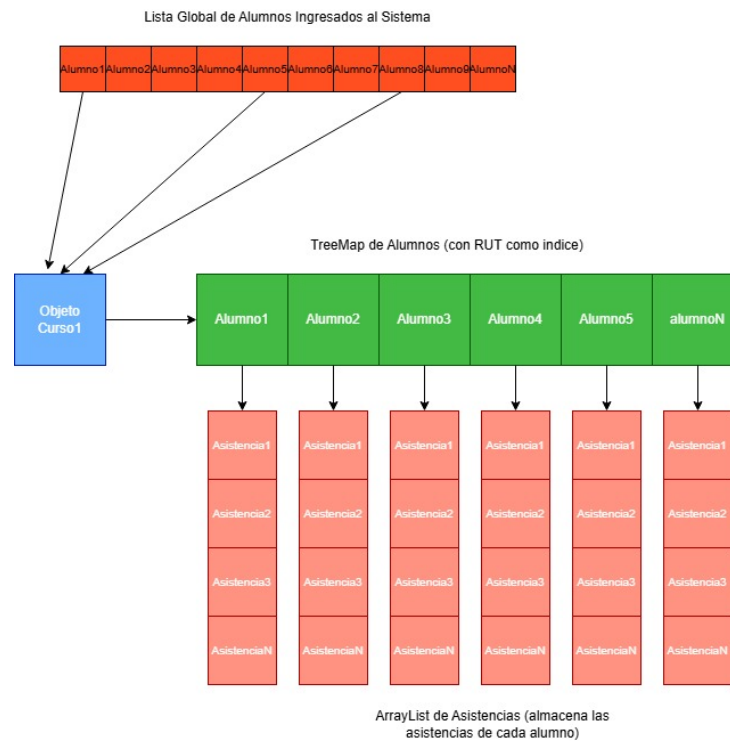


Figura 4.1: Diagrama de Estructuras Principales del sistema

1. Primer nivel – Mapa global de estudiantes:

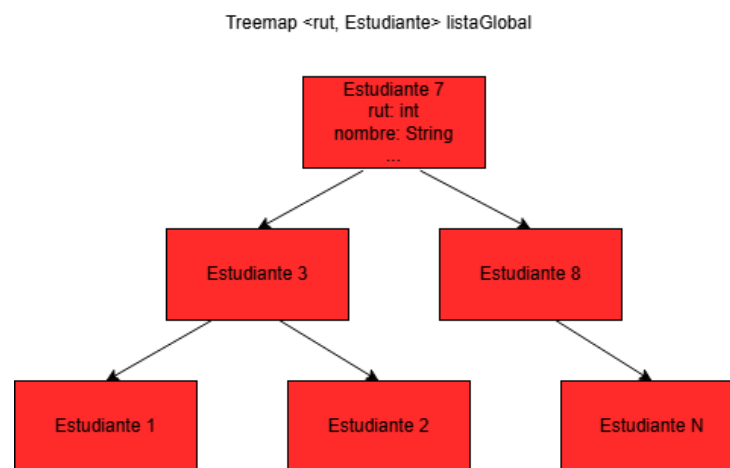


Figura 4.2: Diseño del TreeMap listaGlobal de estudiantes

Es un `TreeMap` donde la **clave** es el RUT (String) y el **valor** es el objeto `Estudiante`. Al usar `TreeMap`, los estudiantes quedan ordenados por RUT automáticamente y podemos encontrar a cualquiera rápidamente.

2. Segundo nivel – Lista de asistencias dentro de cada estudiante:

Estructura de listaAsistencia (ArrayList)

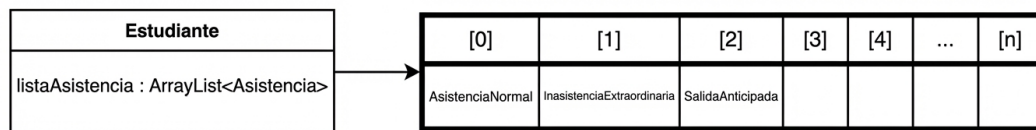


Figura 4.3: Diseño del ArrayList listaAsistencia

Cada `Estudiante` tiene un `ArrayList` que guarda sus registros de asistencia en el orden en que se fueron agregando (cronológico).

Anidamiento: El mapa global contiene estudiantes, y cada estudiante contiene su lista de asistencias. Es una estructura de colección dentro de otra colección".

5. Sobrecarga de métodos (SIA-5)

La sobrecarga consiste en tener varios métodos con el mismo nombre pero distintos parámetros. El compilador decide cuál ejecutar según lo que le pasemos. El sistema implementa sobrecarga en dos clases.

En la clase **Curso**

El método `mostrarEstudiantes()` tiene dos versiones:

- Sin parámetros: muestra todos los estudiantes del curso actual.
- Con un parámetro `String filtroCurso`: filtra y muestra solo los estudiantes cuyo atributo `curso` coincide con el filtro indicado.

En la clase **Estudiante**

El método `agregarAsistencia` presenta dos versiones sobrecargadas:

- Recibe un objeto `Asistencia` ya construido: útil cuando el tipo de asistencia se determina en otra parte del programa (por ejemplo, en los controladores o en la interfaz gráfica). Simplemente añade el objeto a la lista.
- Recibe (`String fecha`, `String observacion`, `boolean puntual`): crea automáticamente una `AsistenciaNormal` con los parámetros indicados. Esta versión es un atajo para el caso más frecuente: registrar que el alumno asistió a clases, indicando si fue puntual o no.

Ambas implementaciones demuestran el uso correcto de la sobrecarga para ofrecer flexibilidad según el contexto de uso.

6. Sobreescritura y polimorfismo (SIA-6)

En el sistema, la clase **Asistencia** es abstracta y declara el método `getResumen()` sin implementación. Esto obliga a cada subclase a definir su propia versión, lo que se conoce como **sobreescritura**. Cada tipo de registro proporciona un mensaje distinto:

- **AsistenciaNormal**: indica si el alumno llegó temprano o no.
- **InasistenciaExtraordinaria**: explica el motivo de la falta.
- **SalidaAnticipada**: detalla la hora de salida.

¿Por qué se diseñó así? En el dominio escolar no existe una asistencia genérica: siempre se trata de un tipo concreto. Al declarar `getResumen()` como abstracto, el sistema garantiza que nunca habrá un registro sin una descripción adecuada a su tipo.

Polimorfismo en acción: al recorrer el historial de un alumno, el sistema simplemente invoca `getResumen()` en cada registro. Java ejecuta automáticamente la versión correspondiente al tipo real de cada objeto, mostrando el mensaje correcto sin necesidad de preguntar de qué tipo es cada asistencia.

7. Insertar y consultar datos del sistema (SIA-7)

El programa permite agregar nuevos elementos y consultar los ya existentes, tanto desde la consola como desde la interfaz gráfica.

Desde la consola

El menú principal ofrece cuatro operaciones básicas:

- **Agregar un estudiante:** se solicitan los datos personales, se validan el RUT y la edad, y el alumno queda registrado en el sistema.
- **Ver todos los estudiantes:** se muestra en pantalla el listado completo de alumnos con su RUT, nombre, curso y edad.
- **Registrar una asistencia:** se elige al alumno por su RUT, se indica la fecha y el tipo de registro (asistencia normal, inasistencia justificada o salida anticipada), y se guarda en su historial.
- **Consultar el historial de un alumno:** se ingresa el RUT y se despliegan todos los registros de asistencia de ese estudiante, con un resumen adaptado a cada tipo.

Desde la interfaz gráfica

En modo ventana, las mismas operaciones se realizan mediante formularios y tablas:

- Un formulario para **ingresar nuevos estudiantes**, con campos de texto y un selector de curso.
- Una **tabla general de estudiantes** que muestra todos los alumnos cargados en el sistema.
- Un panel donde, tras ingresar el RUT, se **visualiza el historial de asistencias** del alumno en una tabla, y desde el cual se puede **registrar una nueva asistencia** mediante cuadros de diálogo.
- Un panel de **asistencia grupal** que permite pasar lista a todo un curso en una sola operación, marcando con casillas los alumnos presentes y registrando automáticamente la asistencia o inasistencia de cada uno.
- Un panel que permite mostrar de manera gráfica la asistencia por alumno y por curso (Esta opción se encuentra solo disponible desde esta interfaz y no por medio del Modo Consola).

En ambos modos, el usuario puede agregar información a las dos colecciones del sistema (estudiantes y asistencias) y consultar su contenido de manera clara y directa.

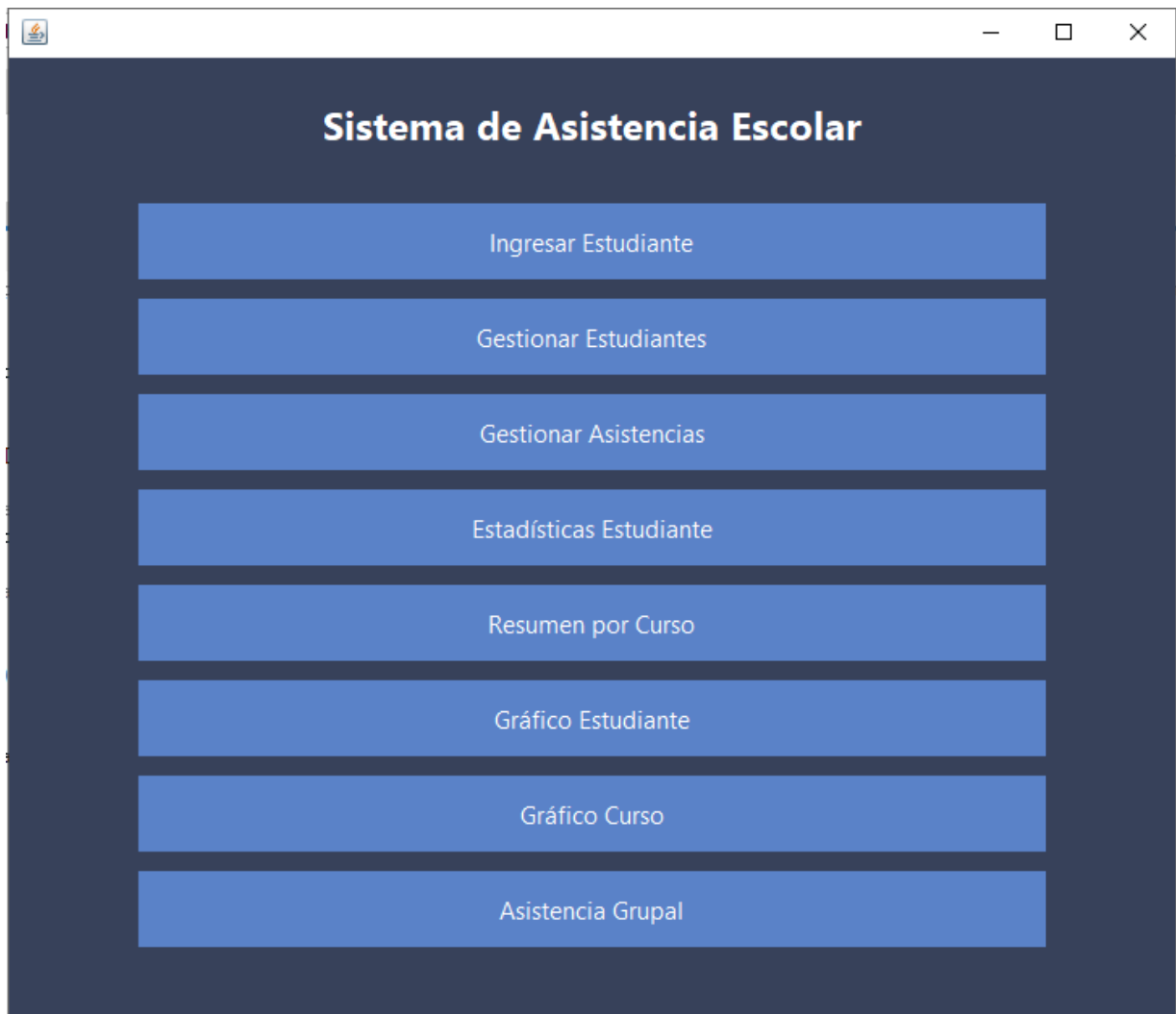


Figura 7.1: Vista previa del menú en modo ventana

8. Modificar, eliminar y buscar información (SIA-8)

Además de agregar y consultar datos, el sistema permite modificar la información existente, eliminar registros que ya no se necesiten y localizar elementos específicos dentro de las colecciones.

Modificar datos

- **Estudiantes:** se puede cambiar el nombre o los apellidos de un alumno ya registrado. En consola se ingresa el RUT del estudiante y se escriben los nuevos datos; en la interfaz gráfica se selecciona al alumno de una tabla y se editan sus campos mediante cuadros de diálogo.
- **Asistencias:** es posible corregir la observación de cualquier registro de asistencia. Tanto en consola como en ventanas se identifica el registro por su código y se escribe la nueva observación.

Eliminar elementos

- **Estudiantes:** se puede borrar un alumno del sistema ingresando su RUT. En la interfaz gráfica se selecciona de una tabla y se confirma la eliminación.
- **Asistencias:** cualquier registro del historial de un alumno puede eliminarse indicando su código único. La acción se confirma antes de ejecutarse para evitar borrados accidentales.

Buscar información

- **Estudiantes:** al ingresar un RUT, el sistema muestra de inmediato el nombre completo, curso, edad y cantidad de asistencias del alumno. En modo gráfico, el resultado aparece en una ventana emergente.
- **Asistencias:** se puede localizar un registro específico dentro del historial de un estudiante mediante su código de identificación. En la interfaz gráfica, la búsqueda se realiza visualmente explorando la tabla de asistencias.

Todas estas operaciones están disponibles tanto en el menú de consola como en los paneles de la interfaz gráfica, ofreciendo al usuario un control completo sobre los datos almacenados en el sistema.

9. Funcionalidad propia del programa (SIA-9)

Además de las operaciones básicas de alta, baja, modificación y consulta, el sistema incluye cinco herramientas pensadas específicamente para facilitar la gestión escolar diaria. Todas aprovechan los datos almacenados en las colecciones para ofrecer información valiosa sin necesidad de escribir código adicional.

Funcionalidad	¿Qué hace?	¿Dónde se encuentra?
Detección de exceso de inasistencias	Recorre todos los estudiantes, cuenta sus inasistencias justificadas y lista los que superan un límite dado por el usuario.	Menú consola (opción 12)
Lista ordenada de un curso	Genera una nómina de estudiantes de un curso, ordenada alfabéticamente por apellidos y nombre.	Menú consola (opción 11)
Estadísticas individuales	Muestra conteos por tipo de asistencia, porcentaje total e indicador de alerta si está bajo 85 %.	Panel Estadísticas Estudiante
Resumen de curso y exportación	Tabla con porcentajes de asistencia de todo un curso, ordenable por columnas y exportable a CSV.	Panel Resumen Curso
Gráficos estadísticos	Gráfico de torta por alumno y gráfico de barras comparativo por curso, con colores de alerta.	Paneles Gráfico Estudiante y Gráfico Curso

Tabla 9.1: Funcionalidades propias del negocio

1. Alumnos con exceso de inasistencias

Esta herramienta ayuda a identificar tempranamente a los estudiantes en riesgo de repitencia por faltas reiteradas. El flujo de trabajo es el siguiente:

¿Cómo detecta el sistema a los alumnos con muchas inasistencias?

1. El usuario ingresa un número máximo de inasistencias permitidas (por ejemplo, 5).
2. El sistema recorre uno por uno a todos los estudiantes registrados en el **TreeMap** global.
3. Para cada estudiante, se invoca un método interno que cuenta cuántos registros de tipo *InasistenciaExtraordinaria* tiene en su historial.
4. Si la cantidad de inasistencias del alumno supera el límite indicado, se muestra en pantalla su nombre completo junto con el número exacto de faltas.
5. Si ningún alumno supera el límite, se informa que no hay casos críticos.

Este proceso no requiere modificar la base de datos ni escribir consultas complejas: simplemente aprovecha la lista de asistencias que ya posee cada estudiante.

2. Generación de lista ordenada por curso

Esta funcionalidad permite obtener rápidamente la nómina de un curso en orden alfabético, ideal para pasar asistencia en clase o elaborar informes oficiales. El procedimiento es igualmente sencillo:

1. El usuario selecciona un curso de la lista predefinida (desde 1º Básico hasta 4º Medio).
2. El sistema filtra a todos los estudiantes cuyo atributo **curso** coincida con el elegido y los coloca en una colección temporal.
3. A continuación, se ordena esa colección utilizando un criterio en cascada: primero por apellido paterno, luego por apellido materno y finalmente por nombre.
4. La lista ordenada se muestra numerada en consola.

El siguiente diagrama de flujo ilustra visualmente los pasos que sigue el método encargado de generar la lista:

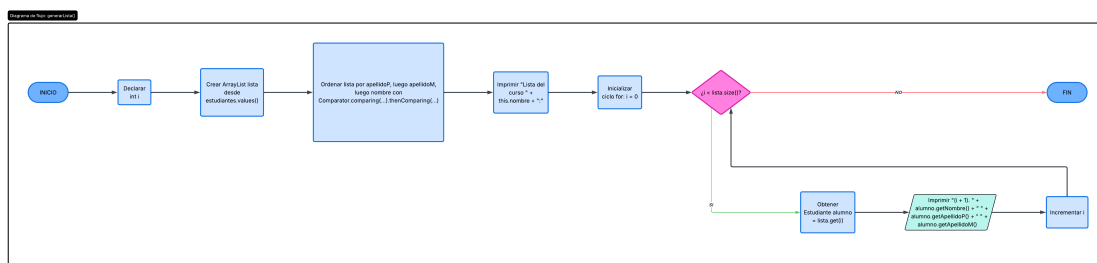


Figura 9.1: Diagrama de flujo del proceso de ordenamiento y visualización de la lista de curso

3. Estadísticas individuales del estudiante

El panel `PanelEstadisticasEstudiante`, disponible en el modo gráfico, presenta un resumen numérico completo del historial de un alumno: cantidad de asistencias normales, inasistencias extraordinarias, salidas anticipadas, total de registros y porcentaje de asistencia. Si el porcentaje es inferior al 85 %, se resalta en rojo como indicador de alerta temprana.

4. Resumen del curso y exportación a CSV

El panel `PanelResumenCurso` genera una tabla con todos los estudiantes de un curso seleccionado, mostrando sus conteos de asistencias por tipo y su porcentaje individual. La tabla puede ordenarse haciendo clic en los encabezados de columna. Además, incluye un botón que guarda la información en un archivo .csv delimitado por punto y coma, compatible con Excel y otras planillas de cálculo.

5. Gráficos estadísticos

Se implementaron dos paneles de visualización gráfica que utilizan `Graphics2D` de Java sin bibliotecas externas:

- **PanelGraficoEstudiante:** dibuja un gráfico de torta con la distribución de asistencias del alumno (normales en verde, inasistencias en rojo, salidas anticipadas en naranja), incluyendo leyenda y porcentajes.
- **PanelGraficoCurso:** dibuja un gráfico de barras que compara el porcentaje de asistencia de todos los alumnos de un curso. Las barras se colorean en azul si el alumno está sobre el 85 %, y en rojo si está por debajo, facilitando la identificación visual de casos críticos.

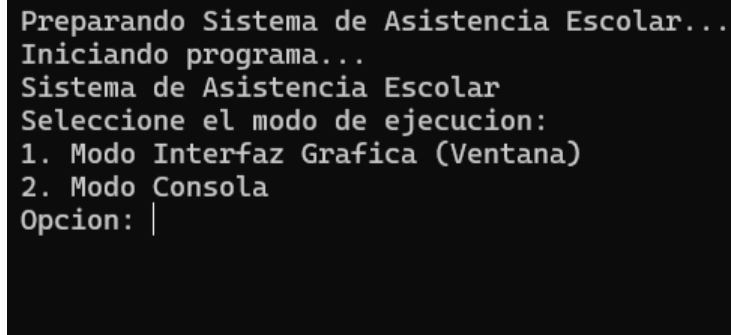
Todas estas funcionalidades demuestran que la estructura de datos anidada del sistema permite generar información útil para la gestión escolar a partir de los registros ya almacenados.

10. Consola o ventanas, a eleccion (SIA-10)

El sistema ofrece al usuario la posibilidad de elegir entre dos modos de ejecución al iniciarse: mediante una **interfaz de consola** (línea de comandos) o una **interfaz gráfica** (ventanas Swing). Ambos modos comparten exactamente la misma lógica.

Selector de modo al arrancar

Al ejecutar el programa, aparece un sencillo menú de selección:



```
Preparando Sistema de Asistencia Escolar...
Iniciando programa...
Sistema de Asistencia Escolar
Seleccione el modo de ejecucion:
1. Modo Interfaz Grafica (Ventana)
2. Modo Consola
Opcion: |
```

Figura 10.1: Selector de modo de arranque

Dependiendo del número ingresado, se inicia la versión correspondiente.

Comparativa de modos

Característica	Modo Consola	Modo Gráfico (Swing)
Entrada de datos	Mediante Scanner (teclado). El usuario escribe comandos numéricos y texto.	Mediante campos de texto, botones, tablas y cuadros de diálogo.
Salida de información	Texto plano en la terminal con <code>System.out.println</code> .	Ventanas, tablas (<code>JTable</code>) y mensajes emergentes (<code>JOptionPane</code>).
Navegación	Menú numérico anidado. Se selecciona una opción y se ejecuta la acción.	Paneles intercambiables con <code>CardLayout</code> y botones para avanzar/retroceder.
Flujo de trabajo	Lineal: el programa espera a que el usuario ingrese un dato antes de continuar.	Orientado a eventos: el usuario dispara acciones al hacer clic en botones o seleccionar filas.
Misma lógica	Usa las clases <code>Main.listaGlobal</code> , <code>Estudiante</code> , <code>Asistencia</code> , etc.	Usa exactamente las mismas clases del modelo.
Persistencia	Al elegir “Guardar y Salir” (opción 0) se llama a <code>guardarEstudiantes()</code> .	Al cerrar la ventana (<code>WindowListener</code>) se invoca automáticamente el mismo método de guardado.

Tabla 10.1: Diferencias y similitudes entre los modos de ejecución

Estructura del modo gráfico

La interfaz Swing se organiza en paneles contenidos en un `JFrame` principal (**Ventana**) que utiliza un `CardLayout` para cambiar entre las siguientes vistas:

- **Menú principal:** botones de acceso a todas las funcionalidades.
- **Ingreso de estudiantes:** formulario con campos de texto y combobox de cursos.
- **Gestión de estudiantes:** tabla con todos los alumnos, botones para buscar, editar y eliminar.
- **Gestión de asistencias:** historial individual con registro, edición y eliminación.
- **Estadísticas de estudiante:** resumen numérico con indicador de alerta.
- **Resumen por curso:** tabla ordenable con exportación a CSV.
- **Gráfico de estudiante:** gráfico de torta de distribución de asistencias.
- **Gráfico de curso:** gráfico de barras comparativo del curso.

- **Asistencia grupal:** pase de lista masivo por curso.

La navegación se controla con el método `cambiarVista(nombre)` y se mantiene un historial para permitir volver a la pantalla anterior.

Esquema del flujo en modo consola

```
Menu Principal
Estudiantes:
  1. Agregar Estudiante
  2. Listar Estudiantes
  3. Buscar Estudiante por RUT
  4. Editar Estudiante
  5. Eliminar Estudiante
Asistencias:
  6. Registrar Asistencia
  7. Ver Historial de Asistencias
  8. Editar Observacion de Asistencia
  9. Eliminar Asistencia
  10. Buscar Asistencia por ID
Reportes:
  11. Generar Lista Ordenada de Curso
  12. Alumnos con exceso de inasistencias
Sistema:
  0. Guardar y Salir
Opcion: |
```

Figura 10.2: Vista del menu en modo consola

Cada opción ejecuta un método estático de la clase `Main` que interactúa con el usuario a través de la consola.

Ventaja del diseño dual

- El **modo consola** es ideal para entornos sin interfaz gráfica, automatización de pruebas o usuarios avanzados que prefieren el teclado.
- El **modo gráfico** resulta más intuitivo para usuarios finales (profesores, administrativos) que esperan una experiencia visual con tablas y botones.
- Ambos modos operan sobre la misma colección de datos (`Main.listaGlobal`), por lo que se puede iniciar en un modo, hacer cambios, cerrar, y luego abrir en el otro modo sin pérdida de información.

11. Los datos no se pierden al cerrar (SIA-11)

El sistema utiliza **dos archivos CSV** en la carpeta `resources/` para conservar la información entre sesiones:

- `estudiantes.csv`: guarda los datos personales (RUT, nombre, apellidos, edad, curso).
- `asistencias.csv`: guarda el historial de asistencias, indicando tipo (NORMAL, INASISTENCIA, SALIDA) y el dato específico de cada una.

Al iniciar, `GestorArchivos.cargarEstudiantes()` lee ambos archivos y reconstruye las colecciones en memoria. Si no existen, se generan automáticamente datos de ejemplo. Al salir —ya sea desde consola o al cerrar la ventana—, `guardarEstudiantes()` escribe el estado actual de todas las colecciones en los mismos archivos, asegurando que nada se pierda.

12. Manejando errores sin que el programa se caiga (SIA-12)

Para evitar que el programa falle ante situaciones como un RUT mal escrito, una edad negativa o un estudiante inexistente, se definieron **tres excepciones personalizadas** y se utilizan bloques `try-catch` tanto en consola como en ventanas.

Excepciones propias del sistema

Excepción	¿Cuándo se lanza?
<code>EstudianteNoEncontradoException</code>	Al buscar un estudiante por RUT y no hallarlo en el mapa global.
<code>RutInvalidoException</code>	Al ingresar un RUT que no cumple con el formato o dígito verificador chileno.
<code>EdadInvalidaException</code>	Al intentar asignar una edad negativa o cero a una persona.

Tabla 12.1: Excepciones personalizadas del sistema

Manejo en diferentes partes del programa

- **Consola (`Main.java`):**
 - Al agregar estudiante: se validan RUT y edad. Si fallan, se lanzan `RutInvalidoException` o `EdadInvalidaException` y se muestra un mensaje específico para cada caso, permitiendo reintentar.
 - Al buscar/editar/eliminar: se captura `EstudianteNoEncontradoException` y se imprime un aviso sin interrumpir el menú.
 - Al leer la edad: se captura `NumberFormatException` para evitar que texto no numérico rompa el flujo.
- **Interfaz gráfica (`Swing`):**
 - En `PanelGestionEstudiantes` y `PanelAsistencias`, si no se encuentra un RUT, se muestra un diálogo con `JOptionPane`.
 - En `IngresoEstudiante`, el botón “Agregar Estudiante” captura `RutInvalidoException`, `EdadInvalidaException` y `NumberFormatException`, mostrando ventanas emergentes y posicionando el cursor en el campo problemático.
- **Persistencia (`GestorArchivos`):**
 - Al leer/escribir los archivos CSV se capturan `IOException` y

`NumberFormatException`, imprimiendo un error en consola pero permitiendo que la aplicación siga funcionando.

Ejemplo representativo

El siguiente esquema resume el flujo típico de manejo de errores al buscar un estudiante:

Búsqueda de estudiante por RUT

1. El usuario ingresa un RUT.
2. Se invoca `Main.obtenerEstudiantePorRut(rut)`.
3. Si el RUT no está en el `TreeMap`, se **lanza** `EstudianteNoEncontradoException`.
4. El bloque `catch` correspondiente **captura** la excepción y muestra un mensaje claro al usuario.
5. El programa continúa su ejecución normal (vuelve al menú o a la pantalla anterior).

13. Control de versiones con GitHub (SIA-13)

El proyecto se gestionó mediante un repositorio en GitHub, accesible en:

<https://github.com/YungRodri/proyectoAsistenciaEscolar>

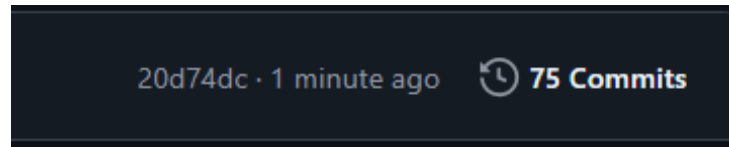


Figura 13.1: Commits realizados en Github

14. Aspectos opcionales implementados

El sistema incorpora cuatro características que van más allá de los requisitos obligatorios y enriquecen la experiencia de uso:

- **Gráficos estadísticos (SIA-01):** el sistema ofrece dos tipos de visualizaciones. Un gráfico de torta que muestra la distribución de asistencias de un alumno (clases normales, inasistencias y salidas anticipadas), y un gráfico de barras que compara el porcentaje de asistencia de todos los alumnos de un curso, destacando en rojo a quienes están por debajo del 85 %.
- **Exportación a planilla de cálculo (SIA-02):** desde la vista de resumen de un curso, se puede descargar toda la información en un archivo CSV compatible con Excel, Google Sheets y otras herramientas de hoja de cálculo. Esto permite elaborar informes externos o compartir los datos con otros docentes.
- **Documentación del código (SIA-03):** las clases y métodos principales incluyen comentarios en formato Javadoc que explican su propósito y funcionamiento. Esto facilita el mantenimiento del sistema y permite generar documentación técnica automática.
- **Organización en capas MVC (SIA-04):** internamente, el código se divide en tres paquetes que separan las responsabilidades: los datos y reglas de negocio (modelo), la interfaz gráfica (vista) y la coordinación entre ambos (controlador). Esta estructura hace que el sistema sea más ordenado y fácil de modificar.